

Tree Decompositions, Treewidth, and Efficient Algorithms for Bounded-Treewidth Graphs

1 Tree Decompositions and Treewidth

1.1 Motivation

Many NP-hard graph problems admit polynomial-time solutions on trees. The reason is simple: a tree has no cycles, so information flows in one direction, and dynamic programming on subtrees works cleanly. *Treewidth* generalises this: it measures how “tree-like” an arbitrary graph is, and algorithms that run in linear time on trees extend to algorithms that run in $O(f(k) \cdot n)$ time on graphs of treewidth k .

1.2 Tree Decompositions

Definition 1.1 (Tree Decomposition). A *tree decomposition* of a graph $G = (V, E)$ is a pair $(T, \{B_t\}_{t \in V(T)})$ where T is a tree and each $B_t \subseteq V$ is called a *bag*, satisfying:

(T1) **Vertex coverage.** $\bigcup_{t \in V(T)} B_t = V$.

(T2) **Edge coverage.** For every $(u, v) \in E$, there exists t with $u, v \in B_t$.

(T3) **Running intersection.** For every $v \in V$, the set $\{t : v \in B_t\}$ induces a connected subtree of T .

The *width* of $(T, \{B_t\})$ is $\max_t |B_t| - 1$. The *treewidth* $\text{tw}(G)$ is the minimum width over all tree decompositions of G .

The -1 convention ensures that trees themselves have treewidth 1 (bags of size 2, one per edge).

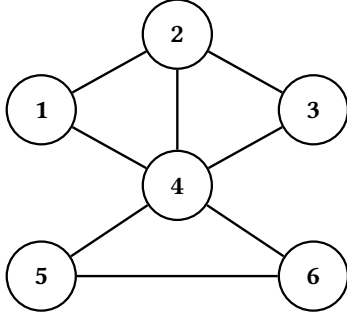
Remark 1.2 (Separator interpretation). Condition (T3) implies that for every node $t \in V(T)$ and every connected component T' of $T - t$, the bag B_t *separates* $\bigcup_{s \in V(T')} B_s \setminus B_t$ from the rest of the graph. This is the key property that allows dynamic programming: once we fix the assignment on B_t , the two sides of the cut are independent.

1.3 A First Example

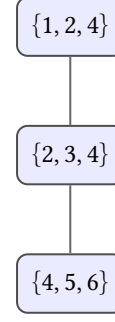
Figure 1 shows the graph G on six vertices $\{1, 2, 3, 4, 5, 6\}$ and a valid tree decomposition of width 2.

Verification.

- **Vertex coverage.** $\{1, 2, 4\} \cup \{2, 3, 4\} \cup \{4, 5, 6\} = \{1, \dots, 6\}$.
- **Edge coverage.** $(1, 2), (1, 4), (2, 4) \subseteq \binom{\{1, 2, 4\}}{2}$; $(2, 3), (3, 4), (2, 4) \subseteq \binom{\{2, 3, 4\}}{2}$; $(4, 5), (4, 6), (5, 6) \subseteq \binom{\{4, 5, 6\}}{2}$.



(a) Graph G.



(b) A tree decomposition of width 2.

Figure 1: A graph G with edges $\{(1, 2), (2, 3), (3, 4), (1, 4), (2, 4), (4, 5), (4, 6), (5, 6)\}$ and a valid tree decomposition.

- **Running intersection.** Vertex 4 appears in all three bags — a connected path. No other vertex spans non-adjacent bags.
- **Width.** $\max(3, 3, 3) - 1 = 2$.

1.4 Key Values and Facts

Graph G	$\text{tw}(G)$	Notes
Tree or forest	1	Bags = $\{u, v\}$ for each edge
Cycle C_n , $n \geq 3$	2	Anchor one vertex in every bag
Series-parallel	≤ 2	Equivalent to no K_4 minor
$m \times n$ grid, $m \leq n$	m	Column-by-column decomposition
Complete graph K_n	$n - 1$	Helly property forces a single n -vertex bag

Theorem 1.3 (Minor monotonicity). *If H is a minor of G then $\text{tw}(H) \leq \text{tw}(G)$.*

Theorem 1.4 (Helly for subtrees). *If $T_1, T_2, \dots, T_m$ are subtrees of a tree T that pairwise intersect, then $\bigcap_{i=1}^m T_i \neq \emptyset$.*

This immediately gives $\text{tw}(K_n) \geq n - 1$: each of the n subtrees T_v pairwise intersects (every pair shares an edge), so all n subtrees share a common bag node, forcing a bag of size $\geq n$.

2 Nice Tree Decompositions

2.1 Definition

For algorithm design it is convenient to work with decompositions in which every transition between adjacent bags changes exactly one vertex.

Definition 2.1 (Nice Tree Decomposition). *A nice tree decomposition is a rooted tree decomposition $(T, \{B_t\})$ in which every node t is one of four types:*

Leaf. t has no children and $|B_t| = 1$.

Introduce node. t has exactly one child c and $B_t = B_c \cup \{v\}$ for some $v \notin B_c$. Vertex v is *introduced* at t .

Forget node. t has exactly one child c and $B_t = B_c \setminus \{v\}$ for some $v \in B_c$. Vertex v is *forgotten* at t .

Join node. t has exactly two children ℓ, r with $B_t = B_\ell = B_r$.

The root has $B_{\text{root}} = \emptyset$.

Theorem 2.2. Any tree decomposition of width k can be converted into a nice tree decomposition of the same width with $O(kn)$ nodes in polynomial time.

2.2 Why Nice Decompositions Speed Up Dynamic Programming

In a general decomposition, moving between adjacent bags of size $k + 1$ with only one vertex overlapping costs $O(4^k)$ per transition (enumerate all assignments on both bags). In a nice decomposition every transition changes exactly one vertex, so each step costs $O(2^k)$, and there are $O(kn)$ steps – giving $O(2^k \cdot kn)$ total, an exponential improvement.

2.3 The Four Node Types Illustrated

Figure 2 shows how each node type modifies the DP table.

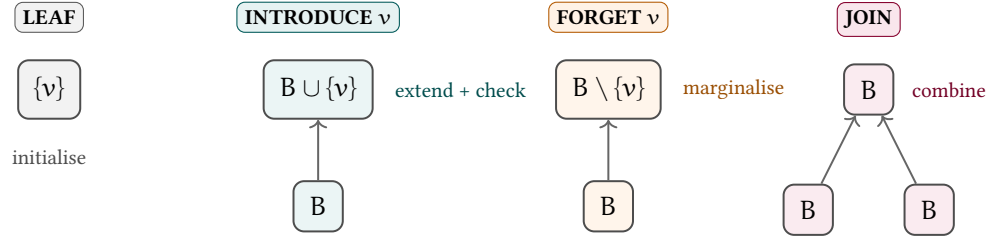


Figure 2: The four node types of a nice tree decomposition. Arrows run from child to parent.

2.4 Edge Activation

A critical property of nice tree decompositions: every edge $(u, v) \in E$ is *activated* at exactly one introduce node – the unique node where both u and v first appear in the same bag. This makes it easy to enforce edge constraints: check them precisely at the introduce node where the edge becomes live, and nowhere else.

2.5 Constructing a Nice Tree Decomposition – Example

Example 2.3. We build a nice tree decomposition of the 4-cycle C_4 on vertices $\{1, 2, 3, 4\}$. Starting from the decomposition $\{1, 2, 4\} - \{2, 3, 4\}$ (width 2), we convert to nice form. The resulting decomposition appears in Figure 3.

3 Series-Parallel Graphs

3.1 Definition via Recursive Composition

Series-parallel graphs are defined recursively by two composition operations on graphs with designated *source* s and *sink* t .

Definition 3.1 (Series-Parallel Graph). A graph G is *two-terminal series-parallel* with terminals (s, t) if it is obtained by the following rules:

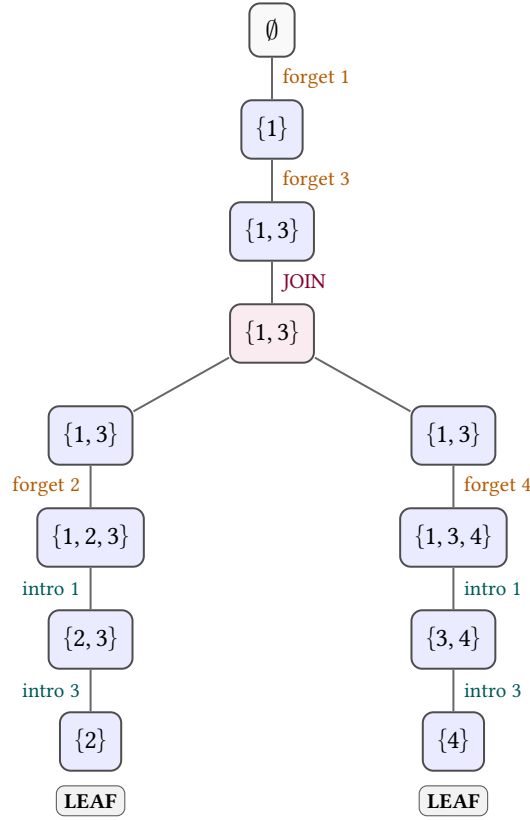


Figure 3: Nice tree decomposition of C_4 with edges $(1, 2)$, $(2, 3)$, $(3, 4)$, $(4, 1)$, reading bottom to top. Edge $(1, 2)$ is activated when 1 is introduced into $\{2, 3\}$; edge $(3, 4)$ is activated when 3 is introduced into $\{4\}$. Width = 2.

(SP1) **Base.** A single edge (s, t) is series-parallel.

(SP2) **Series composition.** If G_1 has terminals (s, w) and G_2 has terminals (w, t) , then the graph obtained by identifying the sink of G_1 with the source of G_2 is series-parallel with terminals (s, t) .

(SP3) **Parallel composition.** If G_1 and G_2 both have terminals (s, t) , then the graph obtained by identifying their sources and their sinks is series-parallel with terminals (s, t) .

Theorem 3.2. A graph G is series-parallel if and only if G contains no K_4 minor. Equivalently, $\text{tw}(G) \leq 2$ if and only if G has no K_4 minor.

3.2 Computing Treewidth Through Composition

The recursive structure of series-parallel graphs gives an immediate inductive proof that $\text{tw} \leq 2$.

Proposition 3.3. Let G be a two-terminal series-parallel graph with terminals (s, t) . Then $\text{tw}(G) \leq 2$, witnessed by a tree decomposition in which every bag contains at most one of $\{s, t\}$ plus at most two other vertices.

Proof. By induction on the construction.

Base case. G is a single edge (s, t) . Use the single bag $B = \{s, t\}$. Width = $1 \leq 2$.

Series composition. Let $G = G_1 \oplus_s G_2$ where G_1 has terminals (s, w) and G_2 has terminals (w, t) . By induction, G_1 has a tree decomposition $\mathcal{T}_1$ of width ≤ 2 where some bag B_1 contains $\{s, w\}$, and G_2 has a tree

decomposition $\mathcal{T}_2$ of width ≤ 2 where some bag B_2 contains $\{w, t\}$. Connect $\mathcal{T}_1$ and $\mathcal{T}_2$ by an edge between B_1 and B_2 . The running intersection property is preserved because w (the shared terminal) appears in both B_1 and B_2 , forming a connected path through the new edge. Width remains ≤ 2 .

Parallel composition. Let $G = G_1 \oplus_p G_2$ where both have terminals (s, t) . By induction, $\mathcal{T}_1$ has a bag $B_1 \supseteq \{s, t\}$ and $\mathcal{T}_2$ has a bag $B_2 \supseteq \{s, t\}$. Connect $\mathcal{T}_1$ and $\mathcal{T}_2$ by an edge between B_1 and B_2 . The running intersection for s and t is preserved through this edge. Width remains ≤ 2 . $\square$

3.3 Illustrated Examples

We trace through three series-parallel graphs and build their tree decompositions by following the recursive construction.

Example 3.4 (Series composition: a path P_4). The path $1 - 2 - 3 - 4$ is the series composition of edges $(1, 2)$, $(2, 3)$, and $(3, 4)$.

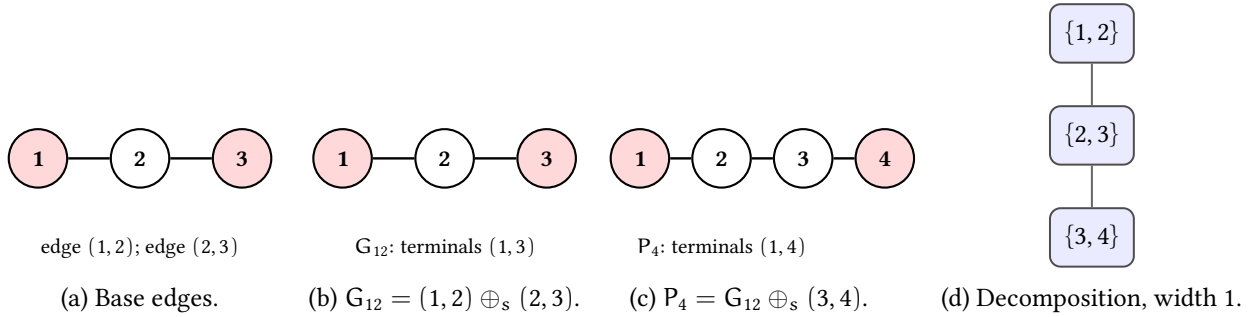


Figure 4: Series composition building P_4 from three base edges. Terminals shown in red.

Example 3.5 (Parallel composition: the theta graph $\Theta(2, 3)$). Two s - t paths — one of length 2 via a , one of length 3 via b, c — are composed in parallel.

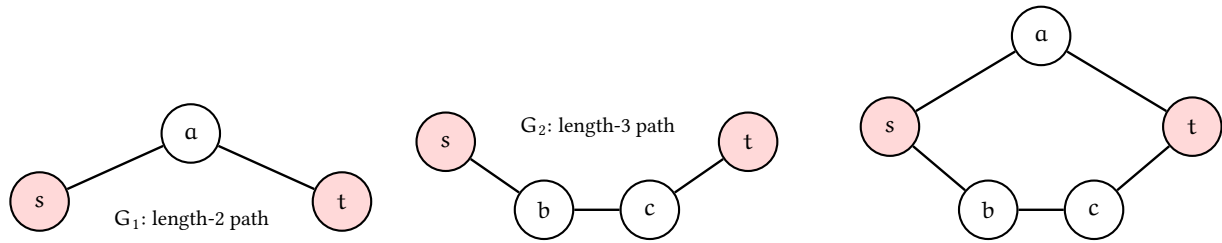


Figure 5: Parallel composition $\Theta(2, 3) = G_1 \oplus_p G_2$. Both G_1 and G_2 have terminals (s, t) .

Decomposition of $\Theta(2, 3)$. G_1 has the decomposition $\{s, a\} - \{a, t\}$ (width 1). G_2 has the decomposition $\{s, b\} - \{b, c\} - \{c, t\}$ (width 1). Both have bags containing $\{s, t\}$ (add $\{s, t\}$ as a join bag). The parallel composition gives the tree in Figure 6.

Example 3.6 (A larger series-parallel graph). Consider the graph H obtained by first composing two edges (s, w) and (w, t) in series, then composing the result in parallel with the edge (s, t) . Figure 7 shows the construction and decomposition.

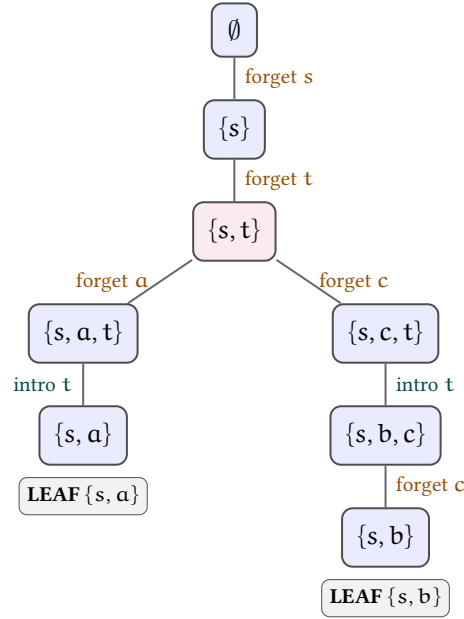


Figure 6: Nice tree decomposition of $\Theta(2, 3)$, width 2. The join bag $\{s, t\}$ witnesses the parallel composition.

3.4 Why Series-Parallel Graphs Have Treewidth Exactly 2 (When Not a Tree)

The lower bound $\text{tw} \geq 2$ for a non-forest series-parallel graph follows from the fact that any cycle C_n (for $n \geq 3$) has $\text{tw} = 2$, and any non-tree graph contains a cycle as a minor, giving $\text{tw} \geq 2$ by minor monotonicity. Combined with Proposition 3.3, this gives $\text{tw} = 2$ for any connected series-parallel graph containing a cycle.

4 Maximum Independent Set on Bounded-Treewidth Graphs

4.1 Setup

Let $G = (V, E)$ have a nice tree decomposition $(T, \{B_t\})$ of width k with $B_{\text{root}} = \emptyset$. Recall: $V_t = \bigcup_{s \in \text{subtree}(t)} B_s$ is the set of all vertices in the subtree rooted at t .

4.2 DP State and Local Validity

State. For each node t and assignment $\sigma : B_t \rightarrow \{0, 1\}$ (where 1 = “in the independent set”):

$$\text{dp}[t][\sigma] = \max\{|S| : S \subseteq V_t, S \text{ independent in } G[V_t], S \cap B_t = \sigma^{-1}(1)\}.$$

Set $\text{dp}[t][\sigma] = -\infty$ if no such S exists.

Call σ *locally valid at t* if $\sigma(u) = \sigma(v) = 1$ for no edge $(u, v) \subseteq B_t$. Any locally invalid σ immediately yields $\text{dp}[t][\sigma] = -\infty$.

4.3 Recurrences

Leaf node. $B_t = \{v\}$.

$$\text{dp}[t][\sigma] = \sigma(v).$$

Both values $\sigma(v) \in \{0, 1\}$ are locally valid (no edges in a single vertex).

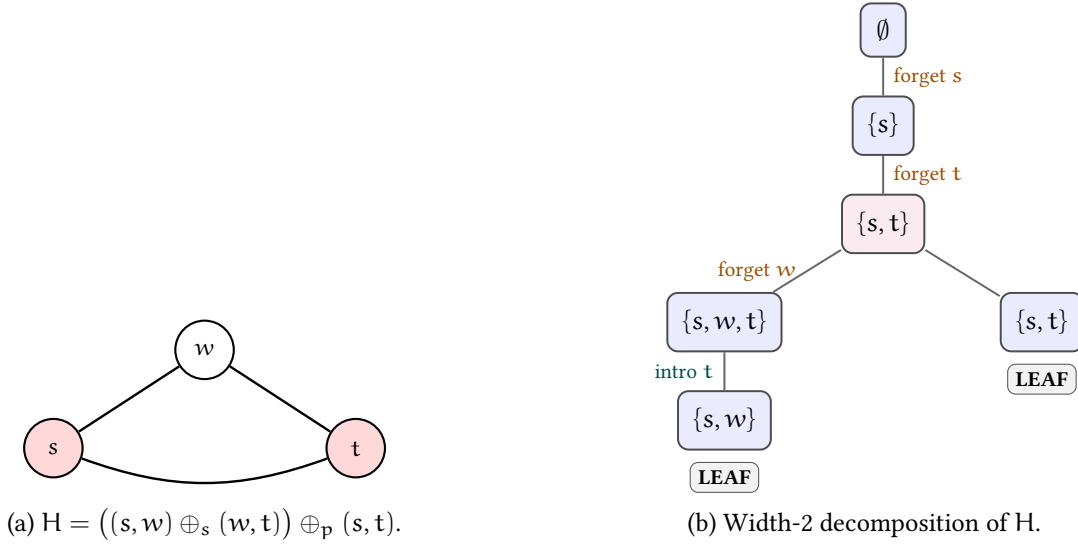


Figure 7: Series-parallel graph H and its tree decomposition.

Introduce node. $B_t = B_c \cup \{v\}$, $\sigma_c = \sigma|_{B_c}$.

$$\text{dp}[t][\sigma] = \begin{cases} -\infty & \text{if } \sigma \text{ not locally valid (i.e. some neighbour } u \in B_c \text{ has } \sigma(u) = \sigma(v) = 1), \\ \text{dp}[c][\sigma_c] + \sigma(v) & \text{otherwise.} \end{cases}$$

Rationale. Vertex v has no neighbours in $V_c \setminus B_c$ (by edge coverage), so the only new constraints are between v and B_c . These are precisely what local validity checks. Adding $\sigma(v)$ accounts for v 's contribution to the IS count.

Forget node. $B_t = B_c \setminus \{v\}$.

$$\text{dp}[t][\sigma] = \max(\text{dp}[c][\sigma \cup \{v \mapsto 0\}], \text{dp}[c][\sigma \cup \{v \mapsto 1\}]).$$

Rationale. After forgetting, v will never interact with vertices introduced later. We take the best possible choice for v 's membership.

Join node. $B_t = B_\ell = B_r$.

$$\text{dp}[t][\sigma] = \text{dp}[\ell][\sigma] + \text{dp}[r][\sigma] - |\sigma^{-1}(1)|.$$

Rationale. $V_\ell \setminus B_t$ and $V_r \setminus B_t$ are disjoint with no cross-edges (structural lemma). Both children count $\sigma^{-1}(1) \cap B_t$ in their scores; subtracting $|\sigma^{-1}(1)|$ removes the double-count.

Answer. $\text{OPT} = \text{dp}[\text{root}][\emptyset]$.

4.4 Correctness Summary

Theorem 4.1. *The DP above correctly computes the maximum independent set of G .*

Proof sketch. By induction on the tree structure. At each node t , $\text{dp}[t][\sigma]$ equals the maximum IS in $G[V_t]$ consistent with σ on B_t . The base case (leaf) holds trivially. Each node type preserves correctness: introduce nodes handle new edges at the right moment; forget nodes marginalise correctly since v has no future interactions; join nodes combine independent subproblems with the exact double-count correction. At the root, $V_{\text{root}} = V$ and $B_{\text{root}} = \emptyset$, so $\text{dp}[\text{root}][\emptyset]$ is the global maximum. $\square$

4.5 Complexity

Theorem 4.2. *The Maximum Independent Set DP runs in $O(2^k \cdot kn)$ time on a graph of treewidth k given its nice tree decomposition, where $n = |V|$.*

Proof sketch. The nice tree decomposition has $O(kn)$ nodes. At each node, the DP table has at most 2^{k+1} entries. Each entry costs $O(k)$ to compute (local validity check requires scanning at most k neighbours). Total: $O(kn) \cdot O(k \cdot 2^k) = O(k^2 \cdot 2^k \cdot n)$. With a more careful precomputation of $|\sigma^{-1}(1)|$ and local validity, one factor of k is absorbed, giving $O(k \cdot 2^k \cdot n)$. $\square$

4.6 Worked Example: MIS on the Theta Graph

We run the IS DP on $\Theta(2, 3)$ using the nice tree decomposition from Figure 6. Vertices: $\{s, a, b, c, t\}$; edges: $(s, a), (a, t), (s, b), (b, c), (c, t)$.

Following the tree bottom-up, the key tables are shown in Figure 8.

Leaf $\{s, a\}$ (after intro a):			Leaf $\{s, b\}$ (after intro b):			Join at $\{s, t\}$ (after forget chains):				
$\sigma(s)$	$\sigma(a)$	dp	$\sigma(s)$	$\sigma(b)$	dp	$\sigma(s)$	$\sigma(t)$	left	right	dp
0	0	0	0	0	0	0	0	2	2	$4 - 0 = 4$
0	1	1	0	1	1	0	1	1	1	$2 - 1 = 1$
1	0	1	1	0	1	1	0	1	1	$2 - 1 = 1$
1	1	$-\infty$	1	1	$-\infty$	1	1	$-\infty$	$-\infty$	$-\infty$

(a) After introducing a into $\{s\}$. Edge (s, a) is activated. (b) After introducing b into $\{s\}$. Edge (s, b) is activated. (c) Join combines left (via a) and right (via b, c). Subtract $|\sigma^{-1}(1)|$.

Figure 8: Selected DP tables for Maximum Independent Set on $\Theta(2, 3)$. The final answer after forgetting s and t at the root is $\text{OPT} = 2$ (e.g. $\{a, b\}$ or $\{a, c\}$ or $\{b, t\}$, etc.).

5 Minimum Vertex Cover on Bounded-Treewidth Graphs

5.1 Problem and Key Observation

A *vertex cover* of G is a set $C \subseteq V$ such that every edge has at least one endpoint in C . Minimum Vertex Cover (MVC) is NP-hard in general.

Remark 5.1 (Complement relation). S is an independent set if and only if $V \setminus S$ is a vertex cover. Hence $|\text{MVC}| = n - |\text{MIS}|$. This duality means the two DPs are nearly identical, with the feasibility condition flipped.

5.2 DP State and Local Validity

State. For each node t and $\sigma : B_t \rightarrow \{0, 1\}$ ($1 = \text{“in the cover”}$):

$$\text{dp}[t][\sigma] = \min\{|C \cap V_t| : C \supseteq \text{some vertex cover of } G[V_t], C \cap B_t = \sigma^{-1}(1)\}.$$

Set $\text{dp}[t][\sigma] = +\infty$ if no valid cover of $G[V_t]$ is consistent with σ (i.e. σ leaves some edge in $G[B_t]$ uncovered).

σ is *locally valid* at t if every edge $(u, v) \subseteq B_t$ has $\sigma(u) = 1$ or $\sigma(v) = 1$.

5.3 Recurrences

Leaf node. $B_t = \{v\}$.

$$\text{dp}[t][\sigma] = \sigma(v).$$

Both assignments are locally valid (no edges in a single vertex).

Introduce node. $B_t = B_c \cup \{v\}$, $\sigma_c = \sigma|_{B_c}$.

$$\text{dp}[t][\sigma] = \begin{cases} +\infty & \text{if } \exists u \in B_c : (u, v) \in E \text{ and } \sigma(u) = \sigma(v) = 0 \text{ (uncovered edge),} \\ \text{dp}[c][\sigma_c] + \sigma(v) & \text{otherwise.} \end{cases}$$

The new constraint is that every newly activated edge (v, u) must be covered: at least one of $\sigma(v), \sigma(u)$ equals 1.

Forget node. $B_t = B_c \setminus \{v\}$.

$$\text{dp}[t][\sigma] = \min\left(\text{dp}[c][\sigma \cup \{v \mapsto 0\}], \text{dp}[c][\sigma \cup \{v \mapsto 1\}]\right).$$

We take the minimum (MVC minimises) over both choices for v .

Join node. $B_t = B_\ell = B_r$.

$$\text{dp}[t][\sigma] = \text{dp}[\ell][\sigma] + \text{dp}[r][\sigma] - |\sigma^{-1}(1)|.$$

The formula is *identical* to MIS. Both children count $\sigma^{-1}(1) \cap B_t$; we subtract once to correct the double-count.

Answer. $\text{OPT} = \text{dp}[\text{root}][\emptyset]$.

5.4 Comparison of MIS and MVC DPs

Aspect	MIS	MVC
Objective	Maximise	Minimise
DP value type	Integer ($-\infty$ for infeasible)	Integer ($+\infty$ for infeasible)
Local validity at t	No two adjacent selected	Every edge within B_t is covered
Leaf	$\text{dp} = \sigma(v)$	$\text{dp} = \sigma(v)$
Introduce: valid case	$\text{dp}[c][\sigma_c] + \sigma(v)$	$\text{dp}[c][\sigma_c] + \sigma(v)$
Introduce: invalid when	$\sigma(u) = \sigma(v) = 1$ for some edge	$\sigma(u) = \sigma(v) = 0$ for some edge
Forget	max over two child entries	min over two child entries
Join	$\text{dp}[\ell] + \text{dp}[r] - \sigma^{-1}(1) $	$\text{dp}[\ell] + \text{dp}[r] - \sigma^{-1}(1) $
Complexity	$O(2^k \cdot kn)$	$O(2^k \cdot kn)$

5.5 Correctness and Complexity

Theorem 5.2. *The MVC DP correctly computes the minimum vertex cover of G in $O(2^k \cdot kn)$ time, where $k = \text{tw}(G)$ and $n = |V|$.*

Both MIS and MVC are fixed-parameter tractable (FPT) in treewidth: for any fixed k , they run in $O(n)$ time.

5.6 Worked Example: MVC on C_4

We run the MVC DP on C_4 with edges $(1, 2), (2, 3), (3, 4), (4, 1)$, using the nice tree decomposition of Figure 3. Table 1 shows the key join step.

Table 1: Join node at $\{1, 3\}$ for MVC on C_4 . Left branch covers path $1 - 2 - 3$; right branch covers path $3 - 4 - 1$. Correction term = $|\sigma^{-1}(1)|$.

$\sigma(1)$	$\sigma(3)$	Left	Right	$ \sigma^{-1}(1) $	Join dp
0	0	1	1	0	$1 + 1 - 0 = 2$
0	1	1	1	1	$1 + 1 - 1 = 1$
1	0	1	1	1	$1 + 1 - 1 = 1$
1	1	2	2	2	$2 + 2 - 2 = 2$

After forgetting 3 (take min over $\sigma(3) = 0, 1$) and then forgetting 1:

$$\text{OPT} = \text{dp}[\text{root}][\emptyset] = \min(1, 1) = \boxed{2}.$$

Indeed, $\{2, 4\}$ is a minimum vertex cover of C_4 (every edge $(1, 2), (2, 3), (3, 4), (4, 1)$ has at least one endpoint in $\{2, 4\}$).

6 Summary

Concept	Key point
Tree decomposition	Organises a graph into bags on a tree; the width is $\max B_t - 1$.
Treewidth	Minimum width over all decompositions; measures how tree-like the graph is.
Nice tree decomposition	Standardises to four node types; reduces DP cost to $O(2^k \cdot kn)$.
Series-parallel graphs	Defined by series and parallel composition; equivalent to $\text{tw} \leq 2$ and no K_4 minor. Treewidth follows inductively from the composition structure.
MIS DP	State = best IS consistent with σ ; forget uses max; join adds and corrects.
MVC DP	State = smallest cover consistent with σ ; forget uses min; join formula is identical to MIS.
FPT in tw	Both MIS and MVC run in $O(2^k \cdot kn)$ — linear in n for fixed k .